Chapter 21

Agentic Environments and Benchmarks

21.1 Motivation: Why Agents Need Environments

The evaluation of a conversational language model is, in principle, straightforward: present a prompt, collect a response, and score it against a reference or via human judgment. Agent evaluation is fundamentally different. An agent must *act* in a world, observe consequences, and adapt its behavior over a sequence of steps. No single response captures this; only a structured *environment* can.

Scope. We use *environment* in the reinforcement-learning sense: a world the agent interacts with for training or evaluation—not the production infrastructure (harness, orchestrator) that hosts the agent at serving time. Execution sandboxes appear here because they *enable* such environments, but the agent harness itself is covered in Chapter 18.

The Chatbot–Agent Evaluation Gap

Chatbot evaluation measures the quality of a single generation: fluency, factuality, helpfulness. **Agent evaluation** measures the quality of a *policy*: does the agent reliably achieve goals across diverse, long-horizon tasks? The gap is not merely quantitative—it requires a different infrastructure.

Three forces drive the need for dedicated agentic environments:

Safe Exploration. Real-world systems—production databases, live websites, financial APIs—cannot absorb the exploratory behavior of an agent under training. A sandboxed environment provides a faithful replica in which the agent can fail, recover, and learn without causing irreversible harm. Security isolation (e.g., Docker containers, network-restricted VMs) is not optional; it is a first-class design requirement.

Reproducible Evaluation. Benchmarking requires that every agent faces the same task under the same conditions. Environments must be deterministic on demand, version-controlled, and distributable so that results reported in one lab can be reproduced in another. The absence of this property has historically made agent benchmarks difficult to compare.

Curriculum Learning. Training an agent from scratch on hard tasks is sample-inefficient. Environments that expose a *difficulty curriculum*—gradually increasing task complexity as the agent improves—dramatically reduce the number of environment interactions required to reach a target performance level. This mirrors how humans learn: mastery of sub-skills precedes mastery of the whole.

Environments as the RL "Gym" for LLMs

Just as OpenAI Gym [31] standardized the interface between RL algorithms and simulated control tasks, agentic environments standardize the interface between LLM-based agents and the diverse tasks they must solve. The analogy is tight: `reset()` initializes a new episode, `step(action)` advances the world and returns an observation and reward, and `render()` produces a human-readable view of the current state.

21.2 Environment Design Principles

A well-designed agentic environment exposes four orthogonal design axes: the *observation space*, the *action space*, the *reward signal*, and the *episode structure*. Getting each right is necessary; getting all four right simultaneously is the craft of environment engineering.

21.2.1 Observation Space Design

The observation is what the agent *sees* at each step. For LLM-based agents the observation is almost always rendered as text, but the source material varies widely:

- **Pure text**: terminal output, file contents, API responses, error messages. Maximally compatible with any LLM but loses spatial and visual structure.
- **Structured (JSON/XML)**: machine-readable state representations. Enables precise grounding but requires the agent to parse structure rather than read prose.
- **Multimodal**: screenshots, accessibility trees, rendered HTML. Necessary for GUI and web tasks; requires a vision-capable model or a separate perception module.
- **Hybrid**: a screenshot paired with an accessibility tree (used in OSWorld and VisualWebArena) gives both visual context and structured element identifiers, combining the strengths of both modalities.

Observation Leakage

A common design mistake is including information in the observation that the agent should not have access to—for example, the ground truth answer, the reward value, or future task steps. Observation leakage inflates benchmark scores and produces agents that fail catastrophically when deployed in real environments where such information is absent.

21.2.2 Action Space Design

The action space defines what the agent can *do*. For LLM agents the action is typically a text string that is parsed and executed by the environment. Common action types include:

- **Tool calls**: structured invocations of external functions (search, calculator, calendar). Often formatted as JSON or XML function-call syntax.
- **Code execution**: the agent writes code that is run in a sandbox; the stdout/stderr is returned as the next observation. This is the most expressive action type.
- **API interactions**: HTTP requests to web services, database queries, shell commands.
- **GUI actions**: `click(x,y)`, `type("text")`, `scroll(direction)`, `key("Enter")`. Used in computer-use environments.
- **Natural language**: free-form text directed at another agent, a human, or a sub-task planner.

21.2.3 Reward Signal Design

Reward design is the hardest part of environment engineering. The reward must be:

1. **Aligned**: high reward should correspond to genuine task completion, not to superficial proxies.
2. **Learnable**: the signal must be dense enough that the agent can make progress; pure sparse rewards on long-horizon tasks are often unlearnable without additional shaping.
3. **Tamper-proof**: the agent must not be able to achieve high reward without actually completing the task (reward hacking).

Table 21.1: Reward signal types for agentic environments with trade-offs.

Reward Type	Pros	Cons
Sparse (0/1 at end)	Aligned, hard to hack	Hard to learn
Dense (step-level)	Easy to learn	Prone to shaping artifacts
Intrinsic (curiosity)	Drives exploration	May diverge from task
LLM-as-judge	Flexible, nuanced	Expensive, inconsistent
Execution-based	Ground truth	Only for verifiable tasks

21.2.4 Episode Structure

Episodes can be structured in several ways:

- **Fixed-length**: the agent takes exactly T steps. Simple to implement; wastes compute on already-solved tasks.
- **Early termination**: the episode ends when the agent signals completion or a terminal state is reached. More efficient but requires a reliable termination detector.
- **Open-ended**: no fixed horizon; the agent operates until a resource budget (tokens, API calls, wall time) is exhausted. Closest to real deployment but hardest to evaluate.

Adaptive episode length and early termination. Recent work challenges the assumption that episode length must be fixed before training begins:

- **Curriculum over horizon.** AELA [411] starts with short episodes and gradually extends the horizon as agent competence grows, measured by policy-entropy convergence. Short early episodes expose more diverse initial states per training sample.
- **Truncation as RL penalty.** DLER [235] shows that the simplest length control—hard truncation—works well for reasoning models when paired with batch-wise reward normalization and dynamic sampling to avoid losing the reward signal from cut-off rollouts.
- **Learned stopping.** Rather than a fixed budget, the model itself can learn when to stop reasoning. [227] propose three strategies: stop when successive reasoning steps converge to the same answer, boost the end-of-thinking token probability, or train a lightweight classifier on hidden-state activations to predict the optimal stopping point.
- **Partial-rollout recycling.** APRIL [251] over-provisions rollout requests and terminates once the target batch count is reached; incomplete responses are recycled as warm-start prefixes in future steps, eliminating the long-tail stall where a few slow samples block the entire batch (20–35% throughput gain). TLT [149] addresses the same bottleneck by training an adaptive draft model on-the-fly for speculative decoding of stragglers (1.7× end-to-end speedup, lossless).

21.2.5 Difficulty Curriculum and Adaptive Environments

Static benchmarks measure a fixed snapshot of agent capability. Adaptive environments go further: they monitor agent performance online and adjust task difficulty to keep the agent in the "zone of proximal development"—hard enough to learn from, easy enough to succeed occasionally. Techniques include:

- **Procedural generation**: tasks are sampled from a parameterized distribution; difficulty parameters are tuned based on recent success rate. Prioritized Level Replay [167] scores each generated level by its estimated learning potential (e.g. GAE magnitude) and replays high-value levels more often.

- **Self-play / adversarial environment design**: PAIRED [74] trains an adversary to propose environments that maximize the *regret* between a protagonist and antagonist agent, producing a natural curriculum of increasing complexity without hand-designed difficulty schedules.
- **Hindsight relabeling**: failed trajectories are relabeled with the goal the agent *did* achieve, providing a learning signal even from failures (Hindsight Experience Replay, HER) [8].
- **Difficulty-targeted data selection for LLMs**: in RLVR training, not all problems provide equal signal. Recent work prioritizes moderate-difficulty questions—those the model solves roughly 30–70% of the time—which yield the highest gradient information [370]. ADCL [230] periodically re-estimates difficulty as the model improves, avoiding stale curricula.

21.3 Types of Agentic Environments

21.3.1 Code Execution Sandboxes

The most fundamental agentic environment for LLMs is a code execution sandbox: the agent writes code, the sandbox runs it, and the output is returned. This simple loop underlies a surprising fraction of real-world agent deployments.

Docker-based isolation is the most common approach. Each episode spawns a fresh container from a known image, executes the agent's code inside it, and destroys the container at episode end. Network access, filesystem writes, and process spawning can all be controlled at the container level.1

E2B (Environments to Benchmarks)2 provides a managed cloud sandbox API: the agent sends code over HTTP, E2B executes it in an isolated Firecracker microVM that boots in under 200 ms, and returns stdout/stderr. E2B handles the infrastructure complexity of container lifecycle management, making it easy to integrate into agent training loops.

Modal3 offers a similar managed execution model with stronger GPU support, making it suitable for agents that need to run ML workloads as part of their task.

Sandbox Escape and Security

Code execution sandboxes are a primary attack surface. A sufficiently capable agent (or a prompt-injected payload) may attempt to escape the sandbox via kernel exploits, network exfiltration, or resource exhaustion. Defense-in-depth is essential: combine container isolation with seccomp profiles, read-only root filesystems, network egress filtering, and CPU/memory cgroups. Never run agent-generated code with host-level privileges.

21.3.2 Web Environments

Web environments present the agent with a browser and ask it to complete tasks on real or simulated websites.

WebArena [440] provides a self-hosted testbed of four functional web applications—an e-commerce store, a social forum, a GitLab instance, and a CMS—plus a map service, totalling 812 long-horizon tasks. The agent interacts via a browser automation API; tasks require multi-step navigation, form filling, and information retrieval. Human performance is approximately 78%; state-of-the-art LLM agents achieve around 35–45%.

VisualWebArena [188] extends WebArena with visually grounded tasks that require interpreting images on web pages. The observation is a screenshot paired with an accessibility tree; the agent must ground its actions in both modalities.

Mind2Web [73] is a large-scale dataset of 2,000 tasks across 137 real websites, collected via human demonstrations. Unlike WebArena, Mind2Web focuses on generalization to unseen websites, making it a harder out-of-distribution test.

WebArena Task Example

Task: "Find the cheapest red dress under $50 on the e-commerce site and add it to the cart."
Agent trajectory:

1. Navigate to the clothing category.
2. Apply color filter: red.
3. Sort by price ascending.
4. Identify the first item under $50.
5. Click "Add to Cart".
6. Verify cart contents.

The environment checks the final cart state against the ground truth item; reward is 1 if correct, 0 otherwise.

21.3.3 Computer Use Environments

Computer use environments give the agent control of a full desktop operating system, observed through screenshots and/or accessibility APIs.

OSWorld [397] tests desktop automation across three operating systems (Ubuntu, Windows, macOS) with 369 tasks spanning productivity apps (LibreOffice, VS Code, Chrome, GIMP, etc.). The agent observes screenshots and acts via `pyautogui`-style mouse and keyboard commands. The human–agent gap is stark: annotators succeed on roughly 72% of tasks while the strongest LLM agent manages only ∼18%, underscoring the difficulty of pixel-level GUI control.

WindowsAgentArena [27] focuses specifically on Windows 11, with 154 tasks across 19 applications. It emphasizes enterprise workflows: Excel formulas, PowerPoint editing, Outlook email management.

The Screenshot Bottleneck

Computer use agents face a fundamental challenge: screenshots are high-dimensional (typically $1920 \times 1080 \times 3$ pixels) but most of the information is irrelevant to the current action. Efficient agents learn to attend to small regions of the screen, use accessibility trees to identify interactive elements by name rather than pixel coordinate, and maintain a compact working memory of previously visited UI states.

21.3.4 Software Engineering Environments

Software engineering (SWE) environments ask the agent to solve real-world programming tasks: fixing bugs, implementing features, writing tests.

SWE-bench [169] draws on 2,294 real pull requests from 12 widely-used Python projects (Django, Flask, scikit-learn, among others). Each instance pairs an issue description with a held-out test suite that passes only after the correct patch is applied. The agent must understand the repository structure, locate the relevant code, implement a fix, and verify it with the test suite. The **SWE-bench Verified** subset (500 issues) has been human-validated for correctness and is the standard evaluation target.

SWE-agent [404] is both a benchmark environment and an agent framework. It introduces the *Agent-Computer Interface* (ACI): a set of shell commands optimized for LLM agents (e.g., `search_file`, `open`, `edit`) that reduce the action space complexity compared to raw bash.

SWE-bench Workflow

Input: A GitHub issue description and the full repository at the commit where the issue was filed.
Agent actions: `find_file`, `view`, `edit`, `python -m pytest tests/`.
Reward: 1 if all target tests pass after the agent's patch; 0 otherwise. No partial credit.

21.3.5 Scientific Research Environments

Scientific research environments push agents toward autonomous knowledge generation: reading papers, forming hypotheses, designing experiments, and interpreting results.

PaperQA2 [245] is a retrieval-augmented agent that answers scientific questions by searching a corpus of PDFs, extracting relevant passages, and synthesizing an answer with citations. It serves as both a tool and a benchmark for literature-grounded reasoning.

AI Scientist [240] is an end-to-end research automation system: given a research direction, the agent generates hypotheses, writes and runs experiments, interprets results, and produces a draft paper. The environment includes a Python execution sandbox, a literature search API, and a LaTeX compiler.

MLAgentBench [151] evaluates agents on machine learning engineering tasks: improving model accuracy on a given dataset within a compute budget. The agent can read data, write training scripts, run experiments, and iterate.

21.3.6 Game and Simulation Environments

Games provide rich, long-horizon environments with well-defined reward signals and no real-world consequences.

NetHack [196] is a procedurally generated roguelike with an enormous state space, requiring long-term planning, inventory management, and adaptation to unexpected events. The NetHack Learning Environment (NLE) provides a Gym-compatible interface.

Voyager / Minecraft [362] uses the Minecraft game engine as an open-ended environment. Voyager introduces a curriculum of progressively harder tasks (collect wood → craft tools → build shelter → explore the Nether) and a skill library that accumulates reusable code snippets across episodes.

GAIA [254] poses 466 questions that demand chained tool use—web search, code execution, file parsing—graded into three difficulty levels by the number of reasoning steps involved. The benchmark starkly exposes the gap between human capability (∼92% accuracy) and current LLM agents (GPT-4 with plugins scored ∼15% at launch; later systems reach ∼30%).

21.3.7 Multi-Agent Environments

Multi-agent environments involve two or more LLM agents interacting with each other and/or a shared world.

- **Negotiation**: agents with private utility functions must reach a deal through dialogue. Classic environments include DealOrNoDeal [206] and CaSiNo [42].
- **Debate**: two agents argue opposing positions; a judge agent (or human) evaluates the quality of arguments. Used to elicit truthful reasoning via adversarial pressure.
- **Collaborative task completion**: agents with complementary capabilities (planner, executor, critic) must coordinate to complete a task neither could solve alone. Frameworks include AutoGen [385], CrewAI [261], and MetaGPT [142].
- **Competitive games**: agents play zero-sum games (chess, Go, poker) where the opponent is itself an LLM agent. Self-play in these environments has produced superhuman performance in narrow domains.

21.4 OpenEnv: Standardized Agentic Environment Interfaces

The proliferation of agentic environments has created a fragmentation problem: each environment exposes a different API, uses different observation formats, and requires different scaffolding. **OpenEnv** [88] is a recent open-source framework by Hugging Face that addresses this directly: it provides

a Gymnasium-style [355] interface (`step()`, `reset()`, `state()`) for agentic execution environments, with isolated Docker-based deployments communicating over WebSocket. OpenEnv complements broader standardization efforts such as AgentGym [390], which offers a uni-format platform for LLM agents across diverse environments, and BrowserGym [201], which standardizes observation and action spaces for web-agent benchmarks. The design principles below capture the converging best practices from these projects.

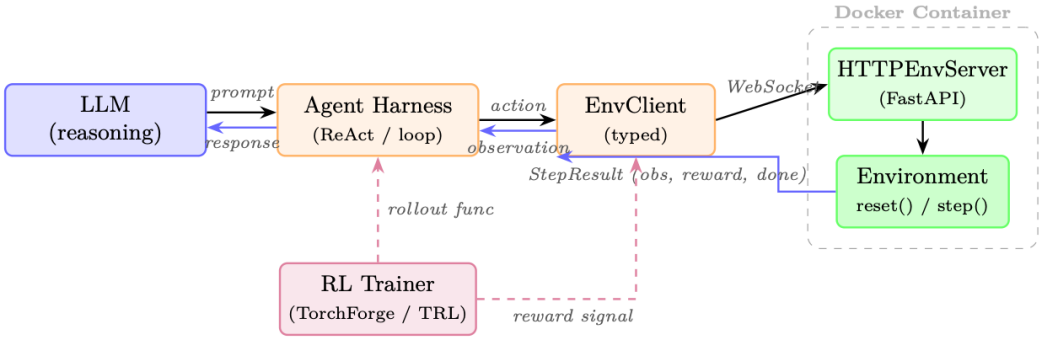

Figure 21.1: OpenEnv architecture with an LLM agent. The agent reasons via a harness loop, which calls the typed `EnvClient`. The client communicates over WebSocket to an `HTTPEnvServer` running inside a Docker container. An RL trainer (dashed) optionally wraps the loop to collect rollouts and reward signals for policy optimization.

21.4.1 Standardized Agent–Environment Interface

OpenEnv defines a typed interface for agentic execution environments. The design mirrors Gymnasium's simplicity but targets LLM agents interacting with tools over HTTP/WebSocket:

- `env.reset()` → `StepResult`: start a new episode; returns the initial observation.
- `env.step(action)` → `StepResult(observation, reward, done)`: execute one action and return the resulting observation, scalar reward, and termination flag.
- `env.state()` → current environment state (episode ID, step count, environment-specific fields).
- `env.close()`: release resources (stop containers, close connections).

Actions and observations are strongly typed Python dataclasses, specific to each environment. For example, a coding environment defines `CodeAction(code=...)` and returns an observation with `stdout`, `stderr`, and `exit_code`; a game environment defines its own action/observation types. This per-environment typing gives agents structured, predictable interfaces while keeping the three core methods (`reset`, `step`, `state`) universal.

Architecture. Each environment is a Python class inheriting from `Environment` (implementing `reset()` and `step()`). It is served inside a Docker container via `HTTPEnvServer`, which exposes a FastAPI/WebSocket endpoint. Clients use environment-specific subclasses of `EnvClient` that handle serialization and connection lifecycle. Containers can be launched locally via `from_docker_image()` or connected to remotely via a base URL:

```
from coding_env import CodeAction, CodingEnv

# Option 1: Launch a local Docker container
client = CodingEnv.from_docker_image("coding-env:latest")

# Option 2: Connect to a remote deployment
# client = CodingEnv(base_url="http://localhost:8000")

# Interact with the environment
result = client.reset()
```

```
print(result.observation.stdout)
print(result.observation.stderr)
print(result.observation.exit_code)

result = client.step(CodeAction(code="print(2 + 2)"))
print(result.observation.stdout)       # "4\n"
print(result.observation.exit_code)    # 0
print(result.reward, result.done)

# Check state
state = client.state()
print(state.episode_id, state.step_count)

client.close()
```

Environment as a server. Creating a new environment requires only implementing the `Environment` base class:

```
from openenv.core.env_server import Environment, create_app
from dataclasses import dataclass

@dataclass
class MyAction:
    text: str

@dataclass
class MyObservation:
    response: str
    reward: float = 0.0
    done: bool = False

class MyEnvironment(Environment):
    def reset(self) -> MyObservation:
        return MyObservation(response="Ready")

    def step(self, action: MyAction) -> MyObservation:
        return MyObservation(response=f"Echo: {action.text}",
                             reward=1.0, done=False)

app = create_app(MyEnvironment(), MyAction, MyObservation)
# Run: uvicorn module:app --host 0.0.0.0 --port 8000
```

Harness integration (experimental). RFC 0054 introduces a harness-facing layer where RL training frameworks interact with environments through MCP-style tool calls. A `build_harness_rollout_func()` helper produces a TRL-compatible rollout function, bridging OpenEnv directly into existing training pipelines like TorchForge [348].

Governance. OpenEnv is openly governed by a technical committee including Meta-PyTorch, NVIDIA, Unsloth, Modal, Prime Intellect, Reflection, and Hugging Face—ensuring that the standard evolves with broad industry input rather than a single vendor's agenda.

21.4.2 Environment Registries and Discovery

OpenEnv environments can be deployed as Hugging Face Spaces or local Docker images, enabling discovery and use without manual installation. The same client interface works regardless of deployment target:

```
from echo_env import EchoAction, EchoEnv

# Connect to a remote HF Space deployment
```

```
client = EchoEnv(base_url="https://openenv-echo-env.hf.space")
result = client.reset()
print(result.observation.echoed_message)  # "Echo environment ready!"

result = client.step(EchoAction(message="Hello!"))
print(result.observation.echoed_message)  # "Hello!"
print(result.reward)
client.close()
```

The OpenEnv ecosystem already spans 70+ environments (OpenSpiel games, Atari, BrowserGym, coding sandboxes, financial RL, traffic simulation, and more). RFC 0025 proposes a formal *tool discovery* protocol so agents can query which actions an unfamiliar environment accepts at runtime.

21.4.3 Compositional Environments

Real agent deployments rarely use a single tool. OpenEnv supports rich environments that expose multiple capabilities through typed actions. For example, a coding environment supports code execution, file I/O, and shell commands within a single sandboxed session:

```
from coding_env import CodeAction, CodingEnv

client = CodingEnv.from_docker_image("coding-env:latest")
result = client.reset()

# Execute code
result = client.step(CodeAction(code="x = 42\nprint(x)"))
print(result.observation.stdout)    # "42"
print(result.observation.exit_code)  # 0

# State persists across steps within an episode
result = client.step(CodeAction(code="print(x + 1)"))
print(result.observation.stdout)    # "43"

state = client.state()
print(state.step_count)  # 2

client.close()
```

For agents requiring diverse tool access (code + web + files), OpenEnv's RFC 0036 proposes MCP integration, allowing any MCP-compatible tool server to be wrapped as an OpenEnv environment. Additionally, the `openenv` CLI can scaffold, build, and deploy new environments to Hugging Face Spaces with a single command.

21.4.4 Environment Versioning and Reproducibility

Benchmark integrity requires that environment behavior is frozen at evaluation time. Best practices include:

- **Semantic versioning**: `WebArena-v1.2.0` guarantees backward compatibility within a minor version.
- **Docker image pinning**: the environment runtime is packaged as a Docker image with a content-addressed hash.
- **Seed-based determinism**: all stochastic elements (procedural generation, network responses) are seeded and logged so that any trajectory can be exactly replayed.
- **Leaderboard snapshots**: public leaderboards record the environment version alongside the score, preventing silent benchmark drift.

21.5 Building Custom Environments

21.5.1 Gymnasium-Style API for LLM Agents

The Gymnasium API [355]7 (successor to OpenAI Gym) is the de facto standard for RL environments. Adapting it for LLM agents requires two modifications: (1) observations and actions are strings (or dicts containing strings) rather than numeric arrays, and (2) the `step` method must handle asynchronous tool execution.

21.5.2 Reward Function Engineering

Reward functions for LLM agent environments are typically *execution-based*: the environment runs a verifier after each episode and returns 1 if the task is solved, 0 otherwise. For tasks without a clear verifier, options include:

- **LLM-as-judge**: a separate LLM scores the agent's final state against the task description.
- **Rubric-based scoring**: a structured rubric decomposes the task into sub-criteria, each scored independently.
- **Human annotation**: a human evaluator scores a random sample of trajectories; the scores are used to calibrate an automated proxy.

21.5.3 State Management and Checkpointing

Long-horizon tasks may require hours of wall time. Environments should support:

- **State serialization**: the full environment state (filesystem, browser cookies, database contents) can be serialized to disk and restored.
- **Mid-episode checkpointing**: the agent can save a checkpoint at any step and resume from it, enabling tree-search-style exploration.
- **Trajectory logging**: every observation, action, and reward is logged to a structured file for offline analysis and reward model training.

21.5.4 Parallelization for Training Data Collection

Training LLM agents via RL requires millions of environment interactions. Parallelization strategies include:

- **Process-level parallelism**: spawn N independent environment processes; collect trajectories in parallel.
- **Async rollout workers**: use an async event loop (e.g., `asyncio`) to overlap LLM inference latency with environment execution.
- **Vectorized environments**: batch multiple environments into a single `step` call, amortizing Python overhead.
- **Cloud-native scaling**: use a job scheduler (Ray, SLURM) to distribute environment workers across a cluster, with a central replay buffer aggregating trajectories.

21.6 Environment–Agent Interface Patterns

Figure 21.2 illustrates the four main interface patterns used in practice.

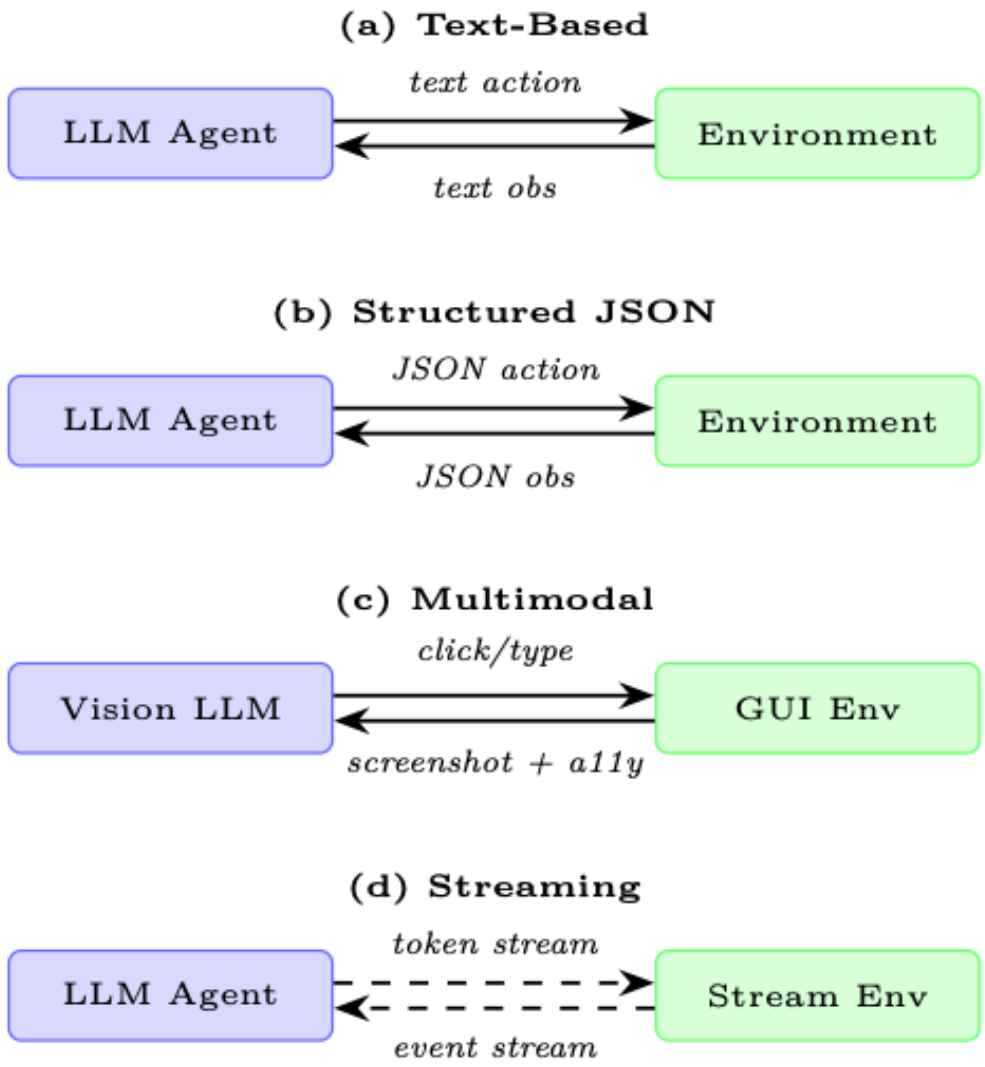

Figure 21.2: Four agent–environment interface patterns. (a) Text-based is the most common for LLMs. (b) Structured JSON enables precise parsing. (c) Multimodal combines screenshots with accessibility trees for GUI tasks. (d) Streaming supports real-time interaction without discrete turn boundaries.

Text-Based Observation/Action. The agent receives a string observation and produces a string action. The environment parses the action (e.g., extracts a tool call from a `<tool>...</tool>` block) and returns the result as a string. This is the most compatible pattern: any LLM can participate without special architecture.

Structured JSON Observation/Action. Observations and actions are JSON objects with a defined schema. This enables strict validation (reject malformed actions before execution), structured logging, and easier programmatic analysis of trajectories. The tradeoff is that the agent must reliably produce valid JSON, which requires either fine-tuning or constrained decoding.

Multimodal (Screenshot + Accessibility Tree). Used in computer-use and web environments. The observation is a tuple `(screenshot: PIL.Image, a11y_tree: dict)`. The screenshot provides visual context; the accessibility tree provides element identifiers that can be used in actions without pixel-level coordinate specification. This hybrid approach is more robust than pure screenshot-based control.

Streaming vs. Turn-Based Interaction. Most current environments use a turn-based model: the agent produces a complete action, the environment executes it, and the next observation is returned. Streaming environments allow the agent to receive partial observations as they arrive (e.g., the output of a long-running command) and to interrupt or redirect execution mid-stream. This is closer to how humans interact with computers but requires more complex agent architectures.

21.7 Evaluation Harness Design

An evaluation harness is the infrastructure that runs an agent across a benchmark suite, collects results, and produces summary statistics. Good harness design is as important as good environment design.

21.7.1 Deterministic vs. Stochastic Environments

- **Deterministic environments** produce the same observation sequence for the same action sequence. They are easy to debug and reproduce but may not reflect real-world variability.
- **Stochastic environments** introduce randomness (procedural generation, network latency, user simulation). They require multiple runs per task to estimate mean performance and confidence intervals.

How Many Runs Are Enough?

For a benchmark with N tasks and binary rewards, the standard error of the mean success rate is $\sqrt{p(1-p)/N}$. With $N = 500$ tasks and $p = 0.4$, the 95% confidence interval is approximately $\pm 4.3\%$. For stochastic environments, multiply by $\sqrt{k}$ where k is the number of independent runs per task. A common practice is 3–5 runs per task for stochastic benchmarks.

21.7.2 Held-Out Test Environments

Benchmark integrity requires a strict train/test split at the *environment* level, not just the task level. An agent that has been trained on WebArena tasks should be evaluated on a held-out set of tasks that were not used during training. Ideally, the held-out set covers different websites, task types, and difficulty levels than the training set.

21.7.3 Cross-Environment Generalization

The ultimate test of an agent is whether skills learned in one environment transfer to another. Cross-environment evaluation protocols measure:

- **Zero-shot transfer**: train on environment A, test on environment B with no fine-tuning.
- **Few-shot adaptation**: provide k demonstrations from environment B before evaluation.
- **Continual learning**: train sequentially on environments A, B, C; measure performance on all three after training on C.

21.7.4 Human Baseline Collection

Every benchmark should include human performance as a reference point. Human baselines serve three purposes:

1. They establish an upper bound on task difficulty.
2. They reveal whether a task is solvable at all (some benchmark tasks turn out to be ambiguous or impossible).

3. They provide a calibration point for interpreting agent scores ("the agent achieves 40% of human performance").

Human baselines should be collected from workers with domain expertise (e.g., software engineers for SWE-bench, not crowdworkers) and should include time-on-task measurements to enable efficiency comparisons.

21.8 Code Example: Minimal Custom LLM Agent Environment

Minimal Custom Environment for LLM Agent Training

The following Python class implements a file-editing environment where the agent must modify a Python file to make a failing test pass. It follows the Gymnasium API adapted for LLM agents.

```
"""
minimal_env.py  --  A minimal file-editing environment for LLM agents.

The agent receives a Python file with a bug and a failing test.
It must edit the file until the test passes.
Reward: 1.0 if all tests pass, 0.0 otherwise.
"""

from __future__ import annotations
import subprocess, shutil, tempfile, textwrap
from pathlib import Path
from dataclasses import dataclass, field
from typing import Any

# ---------------------------------------------------------------------------
# Data structures
# ---------------------------------------------------------------------------

@dataclass
class StepResult:
    observation: str          # Text fed to the LLM
    reward: float             # 0.0 or 1.0
    terminated: bool          # Episode over (task solved or max steps)
    truncated: bool           # Episode cut short (budget exceeded)
    info: dict[str, Any] = field(default_factory=dict)

# ---------------------------------------------------------------------------
# Environment
# ---------------------------------------------------------------------------

class FileEditEnv:
    """
    A Gymnasium-style environment for LLM-based code repair.

    Observation space : str  (file contents + test output)
    Action space      : str  (one of: view, edit, run_tests, submit)
    Reward            : 1.0 on passing all tests, 0.0 otherwise
    """

    MAX_STEPS = 20          # Hard episode limit
    TIMEOUT   = 30          # Seconds per test run

    def __init__(self, buggy_code: str, test_code: str,
                 task_description: str):
        self.buggy_code       = buggy_code
        self.test_code        = test_code
        self.task_description = task_description
        self._workdir: Path | None = None
        self._step_count = 0
```

```
    # ------------------------------------------------------------------
    # Core API
    # ------------------------------------------------------------------

    def reset(self, seed: int | None = None) -> tuple[str, dict]:
        """Initialise a fresh episode; return (observation, info)."""
        if self._workdir and self._workdir.exists():
            shutil.rmtree(self._workdir)

        self._workdir    = Path(tempfile.mkdtemp(prefix="fileenv_"))
        self._step_count = 0

        # Write initial files
        (self._workdir / "solution.py").write_text(self.buggy_code)
        (self._workdir / "test_solution.py").write_text(self.test_code)

        obs = self._build_observation(
            action_taken="[Episode start]",
            test_output=self._run_tests()
        )
        return obs, {"step": 0}

    def step(self, action: str) -> StepResult:
        """Execute one agent action; return StepResult."""
        self._step_count += 1
        action = action.strip()

        # --- Parse and dispatch action ---
        if action.startswith("view"):
            result_text = self._action_view()
        elif action.startswith("edit"):
            result_text = self._action_edit(action)
        elif action.startswith("run_tests"):
            result_text = self._run_tests()
        elif action.startswith("submit"):
            result_text = self._run_tests()
        else:
            result_text = (
                f"Unknown action: {action!r}\n"
                "Valid actions: view | edit <new_content> | "
                "run_tests | submit"
            )

        test_output = self._run_tests()
        passed      = "passed" in test_output and "failed" not in test_output
        reward      = 1.0 if passed else 0.0
        terminated  = passed or action.startswith("submit")
        truncated   = self._step_count >= self.MAX_STEPS

        obs = self._build_observation(action, test_output)
        return StepResult(obs, reward, terminated, truncated,
                          {"step": self._step_count,
                           "passed": passed})

    def render(self) -> str:
        """Return a human-readable summary of the current state."""
        if self._workdir is None:
            return "[Environment not initialised]"
        code = (self._workdir / "solution.py").read_text()
        return f"=== solution.py ===\n{code}\n"

    def close(self) -> None:
        """Release resources."""
        if self._workdir and self._workdir.exists():
            shutil.rmtree(self._workdir)
            self._workdir = None
```

```
    # ------------------------------------------------------------------
    # Private helpers
    # ------------------------------------------------------------------

    def _action_view(self) -> str:
        code = (self._workdir / "solution.py").read_text()
        return f"Current solution.py:\n```python\n{code}\n```"

    def _action_edit(self, action: str) -> str:
        # Expect: edit\n```python\n<code>\n```
        try:
            new_code = action.split("```python")[1].split("```")[0]
            (self._workdir / "solution.py").write_text(new_code)
            return "File updated successfully."
        except IndexError:
            return "Edit failed: wrap new code in ```python ... ```"

    def _run_tests(self) -> str:
        result = subprocess.run(
            ["python", "-m", "pytest", "test_solution.py",
             "-v", "--tb=short", "--no-header"],
            cwd=self._workdir,
            capture_output=True, text=True,
            timeout=self.TIMEOUT
        )
        return result.stdout + result.stderr

    def _build_observation(self, action_taken: str,
                           test_output: str) -> str:
        code = (self._workdir / "solution.py").read_text()
        return textwrap.dedent(f"""
            TASK: {self.task_description}
            STEP: {self._step_count}/{self.MAX_STEPS}

            --- Last action ---
            {action_taken}

            --- Current solution.py ---
            {code}

            --- Test output ---
            {test_output}

            --- Available actions ---
            view                          # show current file
            edit\n```python\n<code>\n```  # replace file contents
            run_tests                     # run pytest
            submit                        # finalise and end episode
        """).strip()

# ------------------------------------------------------------------------
# Example usage
# ------------------------------------------------------------------------

if __name__ == "__main__":
    BUGGY = "def add(a, b):\n    return a - b\n"   # bug: minus not plus
    TESTS = (
        "from solution import add\n"
        "def test_add(): assert add(2, 3) == 5\n"
    )

    env = FileEditEnv(BUGGY, TESTS, "Fix the add() function.")
    obs, _ = env.reset(seed=0)
    print(obs)
```

```
# Simulate one correct edit
fix = "edit\n```python\ndef add(a, b):\n    return a + b\n```"
result = env.step(fix)
print(f"\nReward: {result.reward}  |  Terminated: {result.terminated}")
env.close()
```

Listing 21.1: Minimal LLM agent environment following the Gymnasium API.

Design Decisions in the Example Environment

- **Text-only interface**: observations and actions are plain strings, compatible with any LLM.
- **Execution-based reward**: the reward is derived from running the actual test suite, not from an LLM judge. This makes it tamper-proof and perfectly aligned.
- **Isolated subprocess**: tests run in a separate process with a timeout, preventing infinite loops from crashing the training loop.
- **Gymnasium-compatible**: `reset`/`step`/ `render`/`close` follow the standard API, enabling drop-in use with RL training frameworks.

21.9 Comparison of Major Agentic Environments

Table 21.2 summarizes the key properties of the major agentic environments discussed in this section.

Table 21.2: Comparison of major agentic environments for LLM agents. "SoTA" refers to the best published LLM agent result at the time of writing. Human performance is shown where available.

Environment	Obs. Type	Action Space	Domain	# Tasks	Human	SoTA LLM
WebArena	Text + DOM	Browser API	Web navigation	812	78%	∼45%
VisualWebArena	Screenshot + DOM	Browser API	Visual web	910	88%	∼35%
Mind2Web	Screenshot + DOM	Browser API	Real websites	2,000	—	∼30%
OSWorld	Screenshot	Mouse + keyboard	Desktop OS	369	72%	∼18%
WindowsAgentArena	Screenshot	Mouse + keyboard	Windows apps	154	75%	∼20%
SWE-bench Verified	Text (repo)	Shell + editor	Code repair	500	100%	∼50%
GAIA (Level 1)	Text + files	Tool calls	General QA	165	92%	∼55%
GAIA (Level 3)	Text + files	Tool calls	Hard QA	42	92%	∼10%
NetHack (NLE)	Text + glyphs	Discrete actions	Roguelike game	—	>10k score	∼5k score
Voyager (Minecraft)	Text + code	Code execution	Open-world game	Curriculum	—	15+ tech tree
MLAgentBench	Text + code	Shell + editor	ML engineering	13	—	∼40%

Reading the Comparison Table

The gap between human performance and SoTA LLM performance is largest for *computer use* tasks (OSWorld: 72% vs. 18%) and smallest for *code repair* (SWE-bench: 100% vs. 50%). This pattern reflects the maturity of the action space: LLMs have been trained on vast amounts of code but relatively little screenshot-based interaction data. As computer-use training data accumulates, the gap is expected to narrow.

21.10 Summary

Agentic environments are the substrate on which LLM agents are trained and evaluated. The key takeaways from this section are:

1. **Environments are not optional.** Safe exploration, reproducible evaluation, and curriculum learning all require a structured environment. The gap between chatbot and agent evaluation cannot be bridged without one.
2. **Design all four axes carefully.** Observation space, action space, reward signal, and episode structure each have failure modes that can invalidate an entire benchmark.
3. **The landscape is rich but fragmented.** Code sandboxes, web environments, computer-use environments, SWE environments, scientific environments, games, and multi-agent arenas each test different capabilities. No single environment is sufficient.
4. **Standardization matters.** OpenEnv [88] provides a Gymnasium-style API with Docker isolation and Hugging Face Spaces as a registry—reducing the cost of building new environments and comparing agents across them.
5. **The human gap is real and closing.** Current LLM agents achieve 20–50% of human performance on most benchmarks. The fastest progress is in domains with abundant training data (code) and the slowest in domains requiring fine-grained perception (GUI control).

Open Research Questions in Agentic Environments

- How do we design reward functions for tasks where correctness is subjective or context-dependent?
- Can a single agent architecture generalize across text-based and multimodal environments without task-specific fine-tuning?
- What is the right level of environment fidelity for training? Does training on simplified simulators transfer to real deployments?
- How do we prevent benchmark contamination as LLMs are trained on ever-larger web corpora that may include benchmark solutions?

21.10.1 Emerging Benchmarks (2026)

UniClawBench. UniClawBench [214] runs 400 bilingual real-world tasks inside live, isolated Docker containers—testing agents across five functional areas including cross-platform coordination. Its distinguishing feature is a multi-agent feedback loop: a hidden supervisor and a simulated user inject real-world friction (ambiguous requests, changing requirements, partial failures), measuring *interactive resilience* rather than one-shot task completion. As of mid-2026, it represents the most rigorous benchmark for evaluating production-ready agents.

Long-Horizon-Terminal-Bench. Long-Horizon-Terminal-Bench [386] spans 46 complex tasks across 21 domains, testing whether agents can navigate command-line operations that take hours rather than minutes. Its key innovation is **dense reward-based grading**: scoring intermediate progress at each step rather than relying on binary success at the finish line. This exposes a brutal reality: even the strongest frontier models fail to execute more than ∼15 consecutive terminal commands without derailing—revealing that long-horizon sequential execution remains a fundamental unsolved challenge.